Problem Set 6

Keep your rendered `.pdf` to no more than 5 pages. Only provide code in the rendered `.pdf` when it is specifically requested.

1. Revisit the data on fruit prices from the USDA: https://www.ers.usda.gov/data-products/fruit-and-vegetable-prices. Again, download the file linked below "ALL FRUITS – Average prices (CSV format)." Use the data with the `FruitType` and `Preparation` variables as created as described below:

```
data$FruitType <- unlist(lapply(strsplit(data$Fruit, ", "), function(x) {x[1]}))
data$Preparation <- unlist(lapply(strsplit(data$Fruit, ", "), function(x) {x[2]}))
```

   (a) Create a table of the top 10 most expensive (in terms of average price normalized for quantity as measured by `AveragePricePerCupEquivalent` and averaged over fruit type) fruit types (as measured by `FruitType`) using `kable`. This table should have 10 rows and be arranged in decreasing order of price. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`.

   (b) Create a table of the top 10 least expensive (in terms of average price normalized for quantity as measured by `AveragePricePerCupEquivalent` and averaged over fruit type) fruit types (as measured by `FruitType`) using `kable`. This table should have 10 rows and be arranged in alphabetical order of fruit type. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`.

   (c) Create a table with 7 rows and 5 columns, where each row corresponds to a different type of preparation as measured by `Preparation` (including no information on preparation) and columns correspond to the number of fruits prepared in that way, the average retail price of fruits prepared in that way, the average yield of fruits prepared in that way, the cup equivalent size of fruits prepared in that way, and the cup equivalent price of fruits prepared in that way using `kable`. This table should rows arranged in decreasing order of the number of fruits prepared in each way. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`.

2. Again, find the dataset summarizing some Massachusetts employment statistics called `CESReport.csv`. It was downloaded from here: https://lmi.dua.eol.mass.gov/LMI/CurrentEmploymentStatistics. Use the 170 observations and 15 variables that you obtain after carefully cleaning the data as requested in previous problem set, removing uninformative rows and converting variables to the appropriate type. Note that you can limit the data to just the 170 rows that contain information by specifying the `nrow` argument when you apply the `read.csv` function (although other approaches are also acceptable).

(a) Remove observations from 2025 using `filter`. Print the code you use to do this to the rendered `.pdf`.

(b) Create a new variable called `YTD` which is the sum of the month variables, omitting missing values. Print the code you use to do this to the rendered `.pdf`. Note: You may want to revisit `apply` or `rowSums`.

(c) Using `group_by` and `mutate`, create a new variable called `mYTD` which gives the average value of `YTD` across years within sector, as described by `Description`. Print the code you use to do this to the rendered `.pdf`.

(d) Using `kable`, `arrange`, and `select`, create a table that provides `Year`, `Description`, `YTD`, and `mYTD` for the years and sectors that had the three highest values of `YTD/mYTD`. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`. This table will have a total of 3 rows.

(e) Using `kable`, `arrange`, and `select`, create a table that provides `Year`, `Description`, `YTD`, and `mYTD` for the years and sectors that had the three lowest values of `YTD/mYTD`. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`. This table will have a total of 3 rows.

(f) Explain what you observe in (d) and (e) in at most two sentences.

(g) Using `kable`, `arrange`, and `summarize`, create a table with 1 column and 34 rows that is the sorted list of sectors ordered from most to least variable across years in terms of year to date employment as measured by `YTD`. Make sure that the table is self contained and readable. Print the code you use to do this to the rendered `.pdf`.

(h) Download the additional data on earnings across years and sectors available here: https://download.bls.gov/pub/time.series/ce/ce.data.02b.AllRealEarningsAE. Detailed information about this data is provided here: https://download.bls.gov/pub/time.series/ce/ce.txt. You don't need to get into the weeds understanding it, but you may find it helpful to look at as you go. Read this data into `R` and only keep rows that contain CES" in the `series_id` variable. Print the code you use to do this to the rendered `.pdf`.

(i) The `series_id` variable describes the sector in a more detailed way than our `CES.Series.Code` variable. Later digits describe finer levels of detail, and the first few digits describe the coarse sector definition that matches the level of detail we have in our

original dataset. Using functions for manipulating strings, create a new variable in the data from the previous part called `CES.Series.Code` that matches the first 5 characters of `CES.Series.Code` in our original data, excluding the dash. Print the code you use to do this to the rendered `.pdf`.

(j) Aggregate the new data so that you have one observation per new coarser `CES.Series.Code` and year and the average of `value` across all observations with that `CES.Series.Code` and year. Print the code you use to do this to the rendered `.pdf`.

(k) Aggregate the original data so that you have one observation per `CES.Series.Code` and year and the sum of all `YTD` values across all observations with that `CES.Series.Code` and `Year`. Print the code you use to do this to the rendered `.pdf`.

(l) Merge the two datasets by sector and year, keeping only sectors that appear in both. You may have to modify the `CES.Series.Code` variable in the original dataset. Print the code you use to do this to the rendered `.pdf`.

(m) Aggregate the new merged dataset so that you have one observation per `CES.Series.Code` and the average earnings and jobs variable for each `CES.Series.Code`. Print the code you use to do this to the rendered `.pdf`.

(n) Using the merged data, plot total jobs against average earnings using an x-axis from 0 to $10,000,000$. Using the `text` command in `R`, add text on top of each point that describes the sector, adjusting `cex` to make things fit.